Karnataka Residential Educational Institutions Society

ICT Class 9 - Practical Lab Exam 2

Practical Lab Exam 2 - Answer Key

Class: Class 9

Assessment Type: Practical Lab Exam 2

Total Marks: 20

Duration: 45 minutes

Topics Covered: SQL with Python and Web Development

Answer Key - Grading Guidelines

Note to Teacher: This answer key provides expected answers and marking guidelines. Accept equivalent answers where appropriate. Partial marks may be awarded for incomplete but correct responses.

Lab Tasks (15 marks)

Task 1: SQL Database Operations with Python (5 marks)

- **Answer:**

```
import sqlite3

conn = sqlite3.connect('school.db')
cursor = conn.cursor()

# Create tables
cursor.execute('''CREATE TABLE IF NOT EXISTS Students (
    roll_no INTEGER PRIMARY KEY,
    name TEXT NOT NULL,
    class TEXT,
    section TEXT,
    admission_date DATE
)''')

cursor.execute('''CREATE TABLE IF NOT EXISTS Subjects (
    sub_id INTEGER PRIMARY KEY,
    sub_name TEXT NOT NULL,
    max_marks INTEGER
)''')

cursor.execute('''CREATE TABLE IF NOT EXISTS Results (
    roll_no INTEGER,
    sub_id INTEGER,
    marks_obtained INTEGER,
    FOREIGN KEY (roll_no) REFERENCES Students(roll_no),
    FOREIGN KEY (sub_id) REFERENCES Subjects(sub_id)
)''')

# Insert data
students = [
    (1, 'Amit', '9', 'A', '2024-04-01'),
    (2, 'Priya', '9', 'B', '2024-04-01'),
    (3, 'Rahul', '9', 'A', '2024-04-02'),
    (4, 'Sneha', '9', 'B', '2024-04-02'),
    (5, 'Vikram', '9', 'A', '2024-04-03'),
    (6, 'Ananya', '9', 'B', '2024-04-03'),
]
cursor.executemany('INSERT OR IGNORE INTO Students VALUES (?, ?, ?, ?, ?)', students)

subjects = [(1, 'Mathematics', 100), (2, 'Science', 100), (3, 'English', 100), (4,
'Hindi', 100)]
cursor.executemany('INSERT OR IGNORE INTO Subjects VALUES (?, ?, ?)', subjects)
```

```
results = [
    (1, 1, 85), (1, 2, 78), (1, 3, 92), (1, 4, 88),
    (2, 1, 92), (2, 2, 85), (2, 3, 88), (2, 4, 95),
    (3, 1, 68), (3, 2, 72), (3, 3, 65), (3, 4, 70),
    (4, 1, 88), (4, 2, 91), (4, 3, 85), (4, 4, 82),
    (5, 1, 55), (5, 2, 48), (5, 3, 62), (5, 4, 50),
    (6, 1, 95), (6, 2, 92), (6, 3, 98), (6, 4, 90),
]
cursor.executemany('INSERT OR IGNORE INTO Results VALUES (?, ?, ?)', results)
conn.commit()

# Students with marks above 80
print("=== Students with marks above 80 ===")
cursor.execute('''SELECT s.name, sub.sub_name, r.marks_obtained
    FROM Students s JOIN Results r ON s.roll_no = r.roll_no
    JOIN Subjects sub ON r.sub_id = sub.sub_id
    WHERE r.marks_obtained > 80''')
for row in cursor.fetchall():
    print(f"{row[0]} - {row[1]}: {row[2]}")

# Subject-wise average and highest
print("\n=== Subject-wise Statistics ===")
cursor.execute('''SELECT sub.sub_name, ROUND(AVG(r.marks_obtained),2),
MAX(r.marks_obtained)
    FROM Subjects sub JOIN Results r ON sub.sub_id = r.sub_id
    GROUP BY sub.sub_id''')
for row in cursor.fetchall():
    print(f"{row[0]}: Avg={row[1]}, Highest={row[2]}")

# Students ranked by total marks
print("\n=== Students Ranked by Total ===")
cursor.execute('''SELECT s.name, SUM(r.marks_obtained) as total
    FROM Students s JOIN Results r ON s.roll_no = r.roll_no
    GROUP BY s.roll_no ORDER BY total DESC''')
for i, row in enumerate(cursor.fetchall(), 1):
    print(f"{i}. {row[0]}: {row[1]}")

# Update and compare
print("\n=== Before/After Update ===")
cursor.execute('SELECT marks_obtained FROM Results WHERE roll_no=3 AND sub_id=1')
old = cursor.fetchone()[0]
cursor.execute('UPDATE Results SET marks_obtained=75 WHERE roll_no=3 AND sub_id=1')
conn.commit()
cursor.execute('SELECT marks_obtained FROM Results WHERE roll_no=3 AND sub_id=1')
new = cursor.fetchone()[0]
print(f"Rahul Math: {old} -> {new}")

conn.close()
```

- **Marking:**
  - 1 mark for database connection and table creation with foreign keys
  - 1 mark for inserting student, subject, and result data
  - 1 mark for query: students with marks above 80
  - 1 mark for subject-wise statistics and ranking
  - 1 mark for update with before/after comparison

Task 2: HTML and CSS Web Page (5 marks)

- **Answer:**

```
<!-- school_portal.html -->
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>School Portal</title>
    <link rel="stylesheet" href="portal.css">
</head>
<body>
    <header class="navbar">
        <div class="logo">KREIS School</div>
        <nav class="nav-links">
            <a href="#">Home</a>
            <a href="#">About</a>
            <a href="#">Academics</a>
            <a href="#">Contact</a>
        </nav>
    </header>

    <section class="hero">
        <h1>Welcome to KREIS School</h1>
        <p>Empowering students through quality education</p>
        <button class="cta-btn">Learn More</button>
    </section>

    <section class="students-section">
        <h2>Student Results</h2>
        <table class="student-table">
            <thead>
                <tr><th>Name</th><th>Class</th><th>Section</th><th>Marks</
th><th>Grade</th></tr>
            </thead>
            <tbody>
                <tr><td>Amit</td><td>9</td><td>A</td><td>85</td><td>B</td></tr>
                <tr><td>Priya</td><td>9</td><td>B</td><td>92</td><td>A</td></tr>
                <tr><td>Rahul</td><td>9</td><td>A</td><td>72</td><td>C</td></tr>
                <tr><td>Sneha</td><td>9</td><td>B</td><td>88</td><td>B</td></tr>
                <tr><td>Vikram</td><td>9</td><td>A</td><td>58</td><td>D</td></tr>
                <tr><td>Ananya</td><td>9</td><td>B</td><td>95</td><td>A</td></tr>
            </tbody>
        </table>
    </section>

    <section class="contact-section">
        <h2>Contact Us</h2>
        <form class="contact-form">
            <input type="text" placeholder="Name" required>
            <input type="email" placeholder="Email" required>
            <input type="text" placeholder="Subject" required>
            <textarea placeholder="Message" rows="4" required></textarea>
            <button type="submit">Submit</button>
        </form>
```

```
    </section>

    <footer>
        <p>&copy; 2026 KREIS School. All rights reserved.</p>
    </footer>
</body>
</html>
```

```
/* portal.css */
* { margin: 0; padding: 0; box-sizing: border-box; }
body { font-family: Arial, sans-serif; }

.navbar { display: flex; justify-content: space-between; align-items: center;
padding: 1rem 5%; background: #2c3e50; color: white; }
.logo { font-size: 1.5rem; font-weight: bold; }
.nav-links a { color: white; text-decoration: none; margin-left: 2rem; }
.nav-links a:hover { text-decoration: underline; }

.hero { text-align: center; padding: 4rem 2rem; background: #3498db; color: white; }
.hero h1 { font-size: 2.5rem; margin-bottom: 1rem; }
.cta-btn { padding: 12px 30px; background: #e74c3c; color: white; border: none;
border-radius: 5px; font-size: 1rem; cursor: pointer; }

.students-section { padding: 2rem 5%; }
.student-table { width: 100%; border-collapse: collapse; margin-top: 1rem; }
.student-table th, .student-table td { padding: 12px; border: 1px solid #ddd; text-
align: left; }
.student-table thead { background: #2c3e50; color: white; }
.student-table tbody tr:nth-child(even) { background: #f2f2f2; }
.student-table tbody tr:hover { background: #ddd; }

.contact-section { padding: 2rem 5%; background: #ecf0f1; }
.contact-form { max-width: 500px; display: flex; flex-direction: column; gap: 10px;
margin-top: 1rem; }
.contact-form input, .contact-form textarea { padding: 10px; border: 1px solid #bbb;
border-radius: 4px; }
.contact-form button { padding: 12px; background: #27ae60; color: white; border:
none; border-radius: 4px; cursor: pointer; }

footer { text-align: center; padding: 1rem; background: #2c3e50; color: white; }
```

- **Marking:**
  - 1 mark for header with navigation links
  - 1 mark for hero section with CTA button
  - 1 mark for table with zebra striping and hover effect
  - 1 mark for contact form with all required fields
  - 1 mark for CSS using Flexbox and responsive styling

Task 3: Interactive Web Application (5 marks)

- **Answer:**

```
<!-- todo_app.html -->
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Todo App</title>
```

```
    <style>
        body { font-family: Arial; max-width: 600px; margin: 50px auto; background:
#f5f5f5; }
        .todo-container { background: white; padding: 20px; border-radius: 8px; box-
shadow: 0 2px 10px rgba(0,0,0,0.1); }
        h1 { text-align: center; color: #333; }
        .input-section { display: flex; gap: 10px; margin-bottom: 20px; }
        .input-section input, .input-section select { padding: 10px; border: 1px
solid #ddd; border-radius: 4px; }
        .input-section button { padding: 10px 20px; background: #3498db; color:
white; border: none; border-radius: 4px; cursor: pointer; }
        .task-list { list-style: none; }
        .task-item { display: flex; align-items: center; padding: 12px; margin: 8px
0; border-radius: 4px; background: #f9f9f9; transition: all 0.3s; }
        .task-item:hover { transform: translateX(5px); }
        .task-item.completed span { text-decoration: line-through; color: #999; }
        .priority-high { border-left: 4px solid #e74c3c; }
        .priority-medium { border-left: 4px solid #f39c12; }
        .priority-low { border-left: 4px solid #27ae60; }
        .task-item input[type="checkbox"] { margin-right: 10px; }
        .task-item span { flex: 1; }
        .delete-btn { background: #e74c3c; color: white; border: none; padding: 5px
10px; border-radius: 3px; cursor: pointer; }
        .summary { margin-top: 20px; padding: 15px; background: #ecf0f1; border-
radius: 4px; text-align: center; }
    </style>
</head>
<body>
    <div class="todo-container">
        <h1>Todo List</h1>
        <div class="input-section">
            <input type="text" id="taskInput" placeholder="Enter task">
            <select id="prioritySelect">
                <option value="high">High</option>
                <option value="medium">Medium</option>
                <option value="low">Low</option>
            </select>
            <button onclick="addTask()">Add</button>
        </div>
        <ul class="task-list" id="taskList"></ul>
        <div class="summary" id="summary"></div>
    </div>
    <script>
        var tasks = [];
        function addTask() {
            var title = document.getElementById("taskInput").value.trim();
            var priority = document.getElementById("prioritySelect").value;
            if (!title) return alert("Enter a task!");
            tasks.push({ title: title, priority: priority, completed: false });
            document.getElementById("taskInput").value = "";
            render();
        }
        function toggleTask(index) {
            tasks[index].completed = !tasks[index].completed;
            render();
        }
```

```
        function deleteTask(index) {
            tasks.splice(index, 1);
            render();
        }
        function render() {
            var html = "";
            tasks.forEach(function(task, i) {
                var cls = "task-item priority-" + task.priority + (task.completed ? "
completed" : "");
                html += "<li class='" + cls + "'>" +
                    "<input type='checkbox'" + (task.completed ? " checked" : "") + "
onclick='toggleTask(" + i + ")'>" +
                    "<span>" + task.title + " [" + task.priority + "]</span>" +
                    "<button class='delete-btn' onclick='deleteTask(" + i + ")'>X</
button></li>";
            });
            document.getElementById("taskList").innerHTML = html;
            var total = tasks.length;
            var completed = tasks.filter(function(t) { return t.completed; }).length;
            document.getElementById("summary").innerHTML =
                "Total: " + total + " | Completed: " + completed + " | Pending: " +
(total - completed);
        }
        render();
    </script>
</body>
</html>
```

- **Marking:**
  - 1 mark for task input with priority selection
  - 1 mark for colour-coded priority indicators
  - 1 mark for toggle complete (strikethrough) and delete
  - 1 mark for summary display (total/completed/pending)
  - 1 mark for CSS styling with smooth transitions

Viva Voce (5 marks)

1. **Difference between INNER JOIN and LEFT JOIN.** (1 mark)
   - **Answer:** INNER JOIN returns only rows with matching values in both tables. LEFT JOIN returns all rows from the left table and matching rows from the right table (NULL for non-matching right side).

```
SELECT s.name, r.marks FROM Students s INNER JOIN Results r ON s.roll = r.roll; --
Only students with results
SELECT s.name, r.marks FROM Students s LEFT JOIN Results r ON s.roll = r.roll; --
All students
```

2. **How does CSS specificity work?** (1 mark)
   - **Answer:** CSS specificity determines which rule takes priority: Inline styles (highest) > Internal/ Embedded styles > External stylesheets > Browser defaults (lowest). If two rules have the same specificity, the last one defined wins.

```
p { color: blue; }          /* External - lowest */
<style>p { color: red; }</style>  /* Internal - medium */
<p style="color: green;">    /* Inline - highest */
```

3. **Purpose of a Foreign Key in SQL.** (1 mark)

- **Answer:** A Foreign Key creates a link between two tables by referencing the Primary Key of another table. It maintains referential integrity by ensuring that values in the foreign key column must exist in the referenced table or be NULL.

```
CREATE TABLE Results (
    roll_no INTEGER,
    FOREIGN KEY (roll_no) REFERENCES Students(roll_no)
);
```

4. **Difference between WHERE and HAVING.** (1 mark)
   - **Answer:** `WHERE` filters individual rows before grouping (cannot use aggregate functions). `HAVING` filters groups after `GROUP BY` (can use aggregate functions like COUNT, AVG, SUM).

```
SELECT class, AVG(marks) FROM Students WHERE marks > 50 GROUP BY class HAVING
AVG(marks) > 70;
```

5. **Responsive design vs adaptive design.** (1 mark)
   - **Answer:**
     - Responsive design: Fluid layout that adapts to any screen size using relative units and media queries. Example: CSS Flexbox with percentage widths.
     - Adaptive design: Uses fixed layouts for specific screen sizes (breakpoints). Example: Serving different HTML/CSS for mobile vs desktop.